

Masking.

A single boolean matrix added before the softmax is what separates BERT from GPT. We show how a mask of $-\infty$ entries forces attention weights to exactly zero, derive why this is the right way (not zeroing after softmax), work a full causal example by hand, and trace what it does to the gradients.

THE MECHANISM

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad M_{ij} = \begin{cases} 0 & \text{allowed} \\ -\infty & \text{forbidden.} \end{cases}$$

Worked example. A $T = 4$ self-attention score matrix, masked causally so position t can only see positions $\leq t$. Every weight computed in full.

Topic 5 of 5 in this module · companion to the M0.1 course page · v1.0

Contents

1	What a mask is and why we need one	2
2	Why $-\infty$, and why before the softmax	3
3	Worked example — a causal mask, by hand	3
4	What the mask does to gradients	4
5	Padding masks (the other common case)	5
6	Summary	5
A	Reproducible (NumPy)	5

1 What a mask is and why we need one

The raw attention of Topic 1 lets *every* query attend to *every* key. Often that is wrong:

- A language model predicting the next token must *not* see future tokens — that would be cheating (the answer is in the input).
- A batch of variable-length sentences is padded to equal length; the padding tokens carry no meaning and must be ignored.

A *mask* is an additive matrix M inserted just before the softmax:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right), \quad M_{ij} = \begin{cases} 0 & \text{query } i \text{ may attend to key } j \\ -\infty & \text{forbidden.} \end{cases} \quad (8)$$

Mask	Rule	Used in
none (bidirectional)	every position sees every position	BERT, encoders, ViT
causal (triangular)	position i sees only $j \leq i$	GPT, decoders, LLaMA
padding	ignore [PAD] columns	any variable-length batch

2 Why $-\infty$, and why before the softmax

The trick rests on softmax's exponential. A forbidden score is set to $-\infty$ *before* exponentiating, and

$$e^{-\infty} = 0 \implies A_{ij} = \frac{e^{-\infty}}{\sum_k e^{(\cdot)}} = 0.$$

The forbidden key contributes exactly zero weight, and — crucially — the remaining weights still sum to 1 because the softmax denominator only sums over the surviving terms. The distribution *renormalises automatically* over the allowed keys.

Why not just run a normal softmax and zero out the forbidden entries afterward? Because then the row would no longer sum to 1 — you would have to renormalise by hand, and the zeroed positions would still have leaked into the denominator. Adding $-\infty$ *before* the softmax makes the forbidden keys vanish from the normalisation entirely, which is both mathematically clean and exactly what “this key does not exist for me” should mean. In practice $-\infty$ is implemented as a large negative number (e.g. -10^9), whose exponential underflows to 0.

3 Worked example — a causal mask, by hand

Take a $T = 4$ self-attention score matrix (already scaled), row i = query at position i , column j = key at position j :

$$S = \frac{QK^\top}{\sqrt{d_k}} = \begin{bmatrix} 2.0 & 0.5 & 1.0 & -0.5 \\ 0.3 & 1.8 & 0.7 & 0.2 \\ 1.1 & 0.4 & 2.2 & 0.9 \\ -0.2 & 1.3 & 0.6 & 1.5 \end{bmatrix}.$$

3.1 Bidirectional (no mask) — what BERT does

Softmax each row over all four keys:

$$A_{bi} = \text{softmax}(S) = \begin{bmatrix} 0.598 & 0.133 & 0.220 & 0.049 \\ 0.127 & 0.569 & 0.189 & 0.115 \\ 0.188 & 0.093 & 0.565 & 0.154 \\ 0.076 & 0.340 & 0.169 & 0.415 \end{bmatrix}.$$

Every position mixes information from the whole sequence — including positions to its right.

3.2 Add the causal mask

The causal mask forbids attending to the future: zero on and below the diagonal, $-\infty$ strictly above it.

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad S + M = \begin{bmatrix} 2.0 & -\infty & -\infty & -\infty \\ 0.3 & 1.8 & -\infty & -\infty \\ 1.1 & 0.4 & 2.2 & -\infty \\ -0.2 & 1.3 & 0.6 & 1.5 \end{bmatrix}.$$

3.3 Softmax the masked scores

Each row now normalises only over its allowed (lower-triangular) entries:

$$A_{\text{causal}} = \text{softmax}(S + M) = \begin{bmatrix} 1.000 & 0 & 0 & 0 \\ 0.182 & 0.818 & 0 & 0 \\ 0.222 & 0.110 & 0.668 & 0 \\ 0.076 & 0.340 & 0.169 & 0.415 \end{bmatrix}.$$

Read the structure: position 1 can only attend to itself (1.000). Position 2 splits between tokens 1–2. Position 3 over tokens 1–3. Only the last row, which is already allowed to see everything, equals its bidirectional counterpart. **Every row still sums to 1**, over the past alone.

Causal attention as a heatmap (grey = masked future):

	key1	key2	key3	key4
pos1	1.00	—	—	—
pos2	0.18	0.82	—	—
pos3	0.22	0.11	0.67	—
pos4	0.08	0.34	0.17	0.42

The strictly-lower-triangular shape is the visual signature of an autoregressive model.

4 What the mask does to gradients

Because a masked entry has weight $A_{ij} = 0$ and the softmax Jacobian (Topic 3) is $A_i(\delta - A)$, the masked positions receive **no gradient**: the corresponding score S_{ij} has $\partial L / \partial S_{ij} = 0$. Information flow and gradient flow are both severed in the forbidden direction. Concretely:

- During training, all T positions are processed in parallel, but the causal mask guarantees position t 's loss depends only on tokens $\leq t$ — so the parallel computation matches the sequential one at inference.
- No future token can influence any past token's representation, forward or backward. This is what makes “teacher forcing” valid: the model never learns to rely on information it will not have at generation time.

The mask is the single design choice that defines the model family. BERT-base and GPT-2 small can share *identical* hyperparameters ($d=768$, $h=12$, $L=12$, $d_{\text{ff}}=3072$); the only architectural difference is the mask. No mask $\Rightarrow$ bidirectional context $\Rightarrow$ trained with masked-language-modelling $\Rightarrow$ excels at understanding. Causal mask $\Rightarrow$ left-to-right context $\Rightarrow$ trained with next-token prediction $\Rightarrow$ excels at generation.

5 Padding masks (the other common case)

To batch sentences of different lengths, short ones are padded with [PAD] tokens. Those columns must be masked so no real token attends to padding:

$$M_{ij}^{\text{pad}} = \begin{cases} -\infty & \text{key } j \text{ is [PAD]} \\ 0 & \text{otherwise.} \end{cases}$$

A padding mask zeroes whole *columns* (forbidden keys), whereas a causal mask zeroes an upper *triangle*. They compose by simple addition: $M = M^{\text{causal}} + M^{\text{pad}}$ (a position is allowed only if *both* permit it), which is how a decoder handles padded batches.

Masking the scores but forgetting to mask the *query* rows of pad positions leaves “garbage” outputs at pad positions. That is usually harmless (those outputs are discarded by the loss), but if a later layer pools over all positions — e.g. a mean-pool sentence embedding — the pad outputs leak in. Always exclude pad positions from pooling too.

6 Summary

1. A mask is an additive matrix M applied before the softmax (Eq. 8): 0 for allowed, $-\infty$ for forbidden.
2. $e^{-\infty} = 0$ forces forbidden weights to exactly zero and the row renormalises over survivors — automatically summing to 1.
3. A causal (lower-triangular) mask makes a model autoregressive; a padding mask ignores [PAD]; they add together.
4. Masked positions get zero weight *and* zero gradient, so future tokens never leak forward or backward.
5. Mask choice — bidirectional vs causal — is the defining difference between encoder (BERT) and decoder (GPT) families.

A Reproducible (NumPy)

```
import numpy as np
def sm(a): e=np.exp(a-a.max(1,keepdims=True)); return e/e.sum(1,keepdims=True)
S = np.array([[2.0,0.5,1.0,-0.5],[0.3,1.8,0.7,0.2],
              [1.1,0.4,2.2,0.9],[-0.2,1.3,0.6,1.5]])
mask = np.triu(np.full((4,4), -np.inf), k=1) # -inf strictly above diagonal
print(sm(S)) # bidirectional
print(sm(S + mask)) # causal (lower-triangular weights, rows sum to 1)
```